

**ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**



**BLM4522 - AĞ TABANLI PARALEL DAĞITIM
SİSTEMLERİ**

PROJE - 7

Veritabanı Yedekleme ve Otomasyon Çalışması

Şevval Ensarioğlu - 22290742

GitHub Linki:

<https://github.com/SevvalEnsarioglu/blm4522-proje7-db-backup-automation>

İçindekiler

1 Giriş	3
1.1 Projenin Amacı	3
1.2 Kullanılan Araçlar ve Teknolojiler	4
2 Kullanılan Veritabanı: Northwind	5
2.1 Northwind Tablo Yapısı	5
3 Yedekleme ve Otomasyon Mimarisi	6
3.1 Mimari Bileşenler	6
4 SQL Scriptleri	7
4.1 Şema Oluşturma	7
4.2 Backup Log Tablosu	7
4.3 Backup Audit Tablosu	7
4.4 Backup Alert Tablosu	8
5 Yedekleme Fonksiyonları	9
5.1 Başarılı Backup Log Fonksiyonu	9
5.2 Başarısız Backup Log Fonksiyonu	10
6 Yedekleme Raporlama Yapısı	13
6.1 Backup Rapor View	13
6.2 Günlük Backup Özet View	13
6.3 Açık Uyarılar View	14
7 Yedekleme Otomasyonu	15
7.1 pg_cron Kurulumu	15
7.2 Zamanlanmış Backup Job Örneği	15
7.3 Job Listeleme ve Silme	15
8 Gerçek Backup Alma Komutu	17
9 Test Senaryoları	18
9.1 Başarılı Backup Testi	18

9.2	Başarısız Backup Testi	18
9.3	Rapor Sorguları	18
9.4	Uyarıyı Çözölmüş Olarak İşaretleme	19
10	Proje Kazanımları	20
11	Sonuç ve Değerlendirme	21
11.1	Kaynaklar	21

1 Giriş

Bu proje, BLM4522 Ağ Tabanlı Paralel Dağıtım Sistemleri dersi kapsamında hazırlanmıştır. Proje kapsamında PostgreSQL veritabanı yönetim sistemi üzerinde Northwind örnek veritabanı kullanılarak veritabanı yedekleme, otomasyon, denetim, raporlama ve hata uyarı mekanizması tasarlanmıştır.

Veritabanı yönetiminde yedekleme işlemleri, veri kaybını önlemek ve sistem sürekliliğini sağlamak açısından kritik bir role sahiptir. Manuel olarak yapılan yedekleme işlemleri zamanla unutulabilir, takip edilemeyebilir veya hata durumunda yöneticilere geç bildirim sağlayabilir. Bu nedenle bu projede yedekleme sürecini izleyen, kayıt altına alan, raporlayan ve başarısız işlemler için uyarı oluşturan bir yapı kurulmuştur.

1.1 Projenin Amacı

Bu projenin amacı, veritabanı yedekleme işlemlerini otomatikleştirmek ve yedekleme süreçlerinin düzenli olarak takip edilmesini sağlamaktır.

- PostgreSQL üzerinde yedekleme loglama altyapısı kurmak
- Başarılı ve başarısız yedekleme işlemlerini kayıt altına almak
- Başarısız yedekleme durumlarında uyarı kaydı oluşturmak
- Yedekleme işlemlerini raporlayacak view yapıları hazırlamak
- SQL Server Agent karşılığı olarak PostgreSQL tarafında pg_cron ile otomasyon yapısını göstermek
- Test senaryoları ile başarılı ve başarısız yedekleme durumlarını doğrulamak

1.2 Kullanılan Araçlar ve Teknolojiler

Araç	Sürüm / Tür	Kullanım Amacı
PostgreSQL	Veritabanı Yönetim Sistemi	Northwind veritabanı üzerinde işlemlerin uygulanması
DBeaver	SQL Arayüzü	SQL scriptlerinin çalıştırılması ve sonuçların gözlemlenmesi
Northwind DB	Örnek Veritabanı	Yedekleme senaryosunda kullanılan veritabanı
pg_cron	PostgreSQL Extension	Zamanlanmış otomatik yedekleme işlemlerinin gösterilmesi
pg_dump	PostgreSQL Backup Tool	Gerçek veritabanı yedeği alma komutu
GitHub	Sürüm Kontrolü	Proje dosyalarının ve raporun saklanması

Tablo 1 : Kullanılan araçlar ve teknolojiler

2 Kullanılan Veritabanı: Northwind

Bu projede Northwind örnek veritabanı kullanılmıştır. Northwind, müşteri, ürün, sipariş, çalışan, tedarikçi ve kargo firmaları gibi tablolar içeren klasik bir ilişkisel veritabanıdır. Bu yönüyle yedekleme ve veritabanı yönetimi senaryolarını göstermek için uygun bir yapıya sahiptir.

2.1 Northwind Tablo Yapısı

Tablo	Açıklama
customers	Müşteri bilgilerini tutar
orders	Sipariş başlık bilgilerini tutar
order_details	Sipariş kalemlerini tutar
products	Ürün bilgilerini tutar
categories	Ürün kategori bilgilerini tutar
employees	Çalışan bilgilerini tutar
suppliers	Tedarikçi bilgilerini tutar
shippers	Kargo firmalarını tutar

Tablo 2: Northwind veritabanındaki temel tablolar

3 Yedekleme ve Otomasyon Mimarisi

Bu projede yedekleme işlemlerinin takip edilebilmesi için ayrı bir şema oluşturulmuştur. Tüm log, audit, uyarı ve raporlama yapıları backup_automation şeması altında toplanmıştır.

3.1 Mimari Bileşenler

Bileşen	Görevi
backup_log	Her yedekleme işleminin durumunu, zamanını ve sonucunu kaydeder
backup_audit	Yedekleme sürecinde gerçekleşen olayların denetim kaydını tutar
backup_alerts	Başarısız yedekleme işlemlerinde uyarı kaydı oluşturur
v_backup_report	Yedekleme geçmişini raporlamak için kullanılır
v_daily_backup_summary	Günlük başarılı ve başarısız yedekleme sayılarını gösterir
v_open_backup_alerts	Çözülmemiş yedekleme uyarılarını listeler
pg_cron	Yedekleme işlemlerinin zamanlanmasını sağlar

Tablo 3: Yedekleme otomasyon mimarisi bileşenleri

4 SQL Scriptleri

4.1 Şema Oluşturma

```
CREATE SCHEMA IF NOT EXISTS backup_automation;
```

4.2 Backup Log Tablosu

```
CREATE TABLE IF NOT EXISTS backup_automation.backup_log (  
    backup_id SERIAL PRIMARY KEY,  
    database_name VARCHAR(100) NOT NULL,  
    backup_type VARCHAR(30) NOT NULL,  
    backup_file_path TEXT NOT NULL,  
    backup_status VARCHAR(20) NOT NULL  
        CHECK (backup_status IN ('SUCCESS', 'FAILED', 'STARTED'))  
    ),  
    backup_start_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    backup_end_time TIMESTAMP,  
    duration_seconds NUMERIC(10,2),  
    backup_size_mb NUMERIC(10,2),  
    error_message TEXT  
);
```

Bu tablo, her yedekleme işleminin temel bilgilerini saklamak için oluşturulmuştur. Yedekleme tipi, dosya yolu, işlem durumu, başlangıç zamanı, bitiş zamanı, süre, dosya boyutu ve hata mesajı bu tabloda tutulmaktadır.

4.3 Backup Audit Tablosu

```
CREATE TABLE IF NOT EXISTS backup_automation.backup_audit (  
    audit_id SERIAL PRIMARY KEY,  
    backup_id INT,  
    action_type VARCHAR(50) NOT NULL,  
    action_description TEXT,
```



```

    action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP ,
    performed_by VARCHAR(100) DEFAULT CURRENT_USER ,
    CONSTRAINT fk_backup_audit_log
        FOREIGN KEY (backup_id)
        REFERENCES backup_automation.backup_log(backup_id)
        ON DELETE SET NULL
);

```

Audit tablosu, yedekleme sürecinde gerçekleşen olayları denetim amacıyla kayıt altına alır. Bu sayede hangi işlem, hangi kullanıcı tarafından, ne zaman yapılmış görülebilir.

4.4 Backup Alert Tablosu

```

CREATE TABLE IF NOT EXISTS backup_automation.backup_alerts (
    alert_id SERIAL PRIMARY KEY ,
    backup_id INT ,
    alert_message TEXT NOT NULL ,
    alert_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP ,
    is_resolved BOOLEAN DEFAULT FALSE ,
    resolved_time TIMESTAMP ,
    CONSTRAINT fk_backup_alert_log
        FOREIGN KEY (backup_id)
        REFERENCES backup_automation.backup_log(backup_id)
        ON DELETE SET NULL
);

```

Bu tablo, başarısız yedekleme işlemleri için uyarı kayıtlarını saklar. Gerçek sistemlerde bu yapı e-posta, SMS veya yönetim paneli bildirimi ile genişletilebilir.

5 Yedekleme Fonksiyonları

5.1 Başarılı Backup Log Fonksiyonu

```
CREATE OR REPLACE FUNCTION backup_automation.  
    log_successful_backup(  
        p_database_name VARCHAR,  
        p_backup_type VARCHAR,  
        p_backup_file_path TEXT,  
        p_backup_size_mb NUMERIC DEFAULT NULL  
    )  
RETURNS INT  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_backup_id INT;  
BEGIN  
    INSERT INTO backup_automation.backup_log (  
        database_name ,  
        backup_type ,  
        backup_file_path ,  
        backup_status ,  
        backup_start_time ,  
        backup_end_time ,  
        duration_seconds ,  
        backup_size_mb ,  
        error_message  
    )  
VALUES (  
        p_database_name ,  
        p_backup_type ,  
        p_backup_file_path ,  
        'SUCCESS',  
        CURRENT_TIMESTAMP - INTERVAL '5 seconds',
```

```

        CURRENT_TIMESTAMP,
        5,
        p_backup_size_mb,
        NULL
    )
RETURNING backup_id INTO v_backup_id;

INSERT INTO backup_automation.backup_audit (
    backup_id,
    action_type,
    action_description
)
VALUES (
    v_backup_id,
    'BACKUP_SUCCESS',
    'Veritabanı yedekleme işlemi başarıyla tamamlandı.'
);

RETURN v_backup_id;
END;
$$;

```

5.2 Başarısız Backup Log Fonksiyonu

```

CREATE OR REPLACE FUNCTION backup_automation.log_failed_backup(
    p_database_name VARCHAR,
    p_backup_type VARCHAR,
    p_backup_file_path TEXT,
    p_error_message TEXT
)
RETURNS INT
LANGUAGE plpgsql
AS $$
DECLARE

```

```

        v_backup_id INT;
BEGIN
    INSERT INTO backup_automation.backup_log (
        database_name ,
        backup_type ,
        backup_file_path ,
        backup_status ,
        backup_start_time ,
        backup_end_time ,
        duration_seconds ,
        backup_size_mb ,
        error_message
    )
    VALUES (
        p_database_name ,
        p_backup_type ,
        p_backup_file_path ,
        'FAILED' ,
        CURRENT_TIMESTAMP - INTERVAL '3 seconds' ,
        CURRENT_TIMESTAMP ,
        3 ,
        NULL ,
        p_error_message
    )
    RETURNING backup_id INTO v_backup_id;

    INSERT INTO backup_automation.backup_audit (
        backup_id ,
        action_type ,
        action_description
    )
    VALUES (
        v_backup_id ,
        'BACKUP_FAILED' ,

```

```

        'Veritabanı yedekleme işlemi başarısız oldu.'
    );

    INSERT INTO backup_automation.backup_alerts (
        backup_id,
        alert_message
    )
    VALUES (
        v_backup_id,
        'UYARI: Backup işlemi başarısız oldu. Hata: ' ||
        p_error_message
    );

    RETURN v_backup_id;
END;
$$;

```

Bu fonksiyon, yedekleme işlemi başarısız olduğunda hem log kaydı hem audit kaydı hem de alert kaydı oluşturur. Böylece hata durumu sistem tarafından izlenebilir hale gelir.

6 Yedekleme Raporlama Yapısı

6.1 Backup Rapor View

```
CREATE OR REPLACE VIEW backup_automation.v_backup_report AS
SELECT
    bl.backup_id,
    bl.database_name,
    bl.backup_type,
    bl.backup_status,
    bl.backup_file_path,
    bl.backup_start_time,
    bl.backup_end_time,
    bl.duration_seconds,
    bl.backup_size_mb,
    bl.error_message,
    CASE
        WHEN bl.backup_status = 'SUCCESS' THEN 'Backup
            ba ar l '
        WHEN bl.backup_status = 'FAILED' THEN 'Backup
            ba ar s z '
        ELSE 'Backup ba lad '
    END AS backup_result_description
FROM backup_automation.backup_log bl
ORDER BY bl.backup_start_time DESC;
```

6.2 Günlük Backup Özet View

```
CREATE OR REPLACE VIEW backup_automation.v_daily_backup_summary
AS
SELECT
    DATE(backup_start_time) AS backup_date,
    COUNT(*) AS total_backup_count,
```

```

COUNT(*) FILTER (WHERE backup_status = 'SUCCESS') AS
    successful_backup_count ,
COUNT(*) FILTER (WHERE backup_status = 'FAILED') AS
    failed_backup_count ,
ROUND(AVG(duration_seconds), 2) AS average_duration_seconds
FROM backup_automation.backup_log
GROUP BY DATE(backup_start_time)
ORDER BY backup_date DESC;

```

6.3 Açık Uyarılar View

```

CREATE OR REPLACE VIEW backup_automation.v_open_backup_alerts AS
SELECT
    ba.alert_id ,
    ba.backup_id ,
    bl.database_name ,
    bl.backup_type ,
    ba.alert_message ,
    ba.alert_time ,
    ba.is_resolved
FROM backup_automation.backup_alerts ba
JOIN backup_automation.backup_log bl
    ON ba.backup_id = bl.backup_id
WHERE ba.is_resolved = FALSE
ORDER BY ba.alert_time DESC;

```

7 Yedekleme Otomasyonu

Proje yönergesinde SQL Server Agent kullanılarak yedekleme süreçlerinin otomatikleştirilmesi istenmektedir. Bu proje PostgreSQL üzerinde geliştirildiği için SQL Server Agent yerine PostgreSQL tarafındaki zamanlanmış görev eklentisi olan `pg_cron` kullanılmıştır.

7.1 pg_cron Kurulumu

```
CREATE EXTENSION IF NOT EXISTS pg_cron;
```

7.2 Zamanlanmış Backup Job Örneği

```
SELECT cron.schedule(  
    'daily_northwind_backup_log',  
    '0 2 * * *',  
    $$  
    SELECT backup_automation.log_successful_backup(  
        'northwind',  
        'FULL',  
        '/Users/Shared/backups/northwind_auto_backup_'  
        || TO_CHAR(CURRENT_DATE, 'YYYY-MM-DD') || '.sql',  
        12.50  
    );  
    $$  
);
```

Bu job, her gün saat 02:00'de çalışacak şekilde planlanmıştır. Gerçek yedekleme dosyası terminal üzerinden `pg_dump` ile alınabilir; SQL tarafında ise işlem sonucu loglanarak raporlanabilir hale getirilmiştir.

7.3 Job Listeleme ve Silme

```
SELECT * FROM cron.job;
```



```
SELECT cron.unschedule('daily_northwind_backup_log');
```

8 Gerçek Backup Alma Komutu

PostgreSQL'de gerçek yedekleme işlemi terminal üzerinden `pg_dump` komutu ile yapılır. Aşağıdaki komut Northwind veritabanının SQL formatında yedeğini alır.

```
pg_dump -U postgres -d northwind -F p -f /Users/Shared/backups/northwind_full_backup.sql
```

Eğer PostgreSQL farklı bir port üzerinde çalışıyorsa port bilgisi de eklenebilir:

```
pg_dump -h localhost -p 5432 -U postgres -d northwind -F p -f /Users/Shared/backups/northwind_full_backup.sql
```

Bu komut çalıştırıldıktan sonra başarılı işlem sonucu aşağıdaki fonksiyon ile sisteme kaydedilebilir:

```
SELECT backup_automation.log_successful_backup(  
    'northwind',  
    'FULL',  
    '/Users/Shared/backups/northwind_full_backup.sql',  
    12.50  
);
```

9 Test Senaryoları

9.1 Başarılı Backup Testi

```
SELECT backup_automation.log_successful_backup(  
    'northwind',  
    'FULL',  
    '/Users/Shared/backups/northwind_full_backup.sql',  
    12.50  
);
```

Bu testte sistem başarılı bir yedekleme işlemi yapılmış gibi kayıt oluşturur. Sonuç olarak backup_log ve backup_audit tablolarına kayıt eklenir.

9.2 Başarısız Backup Testi

```
SELECT backup_automation.log_failed_backup(  
    'northwind',  
    'FULL',  
    '/Users/Shared/backups/northwind_failed_backup.sql',  
    'Backup klas r ne eri im izni bulunamad .'   
);
```

Bu testte başarısız bir yedekleme senaryosu oluşturulmuştur. Sonuç olarak backup_log, backup_audit ve backup_alerts tablolarına kayıt eklenir.

9.3 Rapor Sorguları

```
SELECT * FROM backup_automation.v_backup_report;  
  
SELECT * FROM backup_automation.v_daily_backup_summary;  
  
SELECT * FROM backup_automation.v_open_backup_alerts;  
  
SELECT * FROM backup_automation.backup_audit;
```

9.4 Uyarıyı Çözülmüş Olarak İşaretleme

```
UPDATE backup_automation.backup_alerts
SET
    is_resolved = TRUE,
    resolved_time = CURRENT_TIMESTAMP
WHERE alert_id = 1;
```

10 Proje Kazanımları

Bu proje sonucunda PostgreSQL üzerinde yedekleme süreçlerini takip eden ve raporlayan bir otomasyon altyapısı kurulmuştur.

Proje Maddesi	Karşılanan İşlev
Yedekleme otomasyonu	pg_cron ile zamanlanmış backup job yapısı kuruldu
Yedekleme raporları	v_backup_report ve v_daily_backup_summary viewleri oluşturuldu
Başarısız işlem uyarısı	backup_alerts tablosu ve v_open_backup_alerts viewi oluşturuldu
Denetim kayıtları	backup_audit tablosu ile işlem geçmişi tutuldu
Test senaryoları	Başarılı ve başarısız backup kayıtları oluşturuldu

Tablo 4: Proje gereksinimleri ve karşılanan işlevler

11 Sonuç ve Değerlendirme

Bu proje kapsamında Northwind veritabanı üzerinde PostgreSQL kullanılarak yedekleme ve otomasyon odaklı bir veritabanı yönetim altyapısı tasarlanmıştır. Projede manuel yedekleme işlemlerinin izlenebilir hale getirilmesi, başarılı ve başarısız işlemlerin kayıt altına alınması, denetim tablolarının oluşturulması ve raporlama viewlerinin hazırlanması sağlanmıştır.

Başarısız yedekleme işlemleri için uyarı mekanizması kurulmuş ve bu uyarıların çözülüp çözülmediğini takip edebilen bir yapı tasarlanmıştır. Ayrıca SQL Server Agent yerine PostgreSQL ortamına uygun olarak `pg_cron` kullanılarak zamanlanmış yedekleme otomasyonu gösterilmiştir.

Sonuç olarak bu çalışma, veritabanı yönetiminde yedekleme süreçlerinin yalnızca dosya oluşturma işleminden ibaret olmadığını; aynı zamanda loglama, denetim, hata yönetimi, otomasyon ve raporlama gibi bileşenlerle desteklenmesi gerektiğini göstermiştir.

11.1 Kaynaklar

- PostgreSQL Documentation: <https://www.postgresql.org/docs/>
- PostgreSQL pg_dump Documentation: <https://www.postgresql.org/docs/current/app-pgdump.html>
- pg_cron GitHub Documentation: https://github.com/citusdata/pg_cron
- Northwind PostgreSQL: https://github.com/pthom/northwind_psql